



CS 498: Tensor Parallelism

Fan Lai

The Grainger College of Engineering

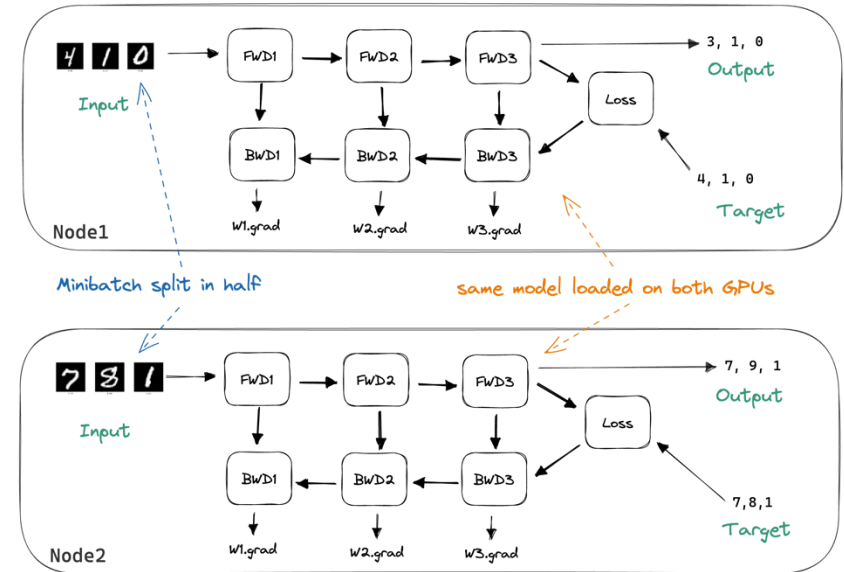
Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

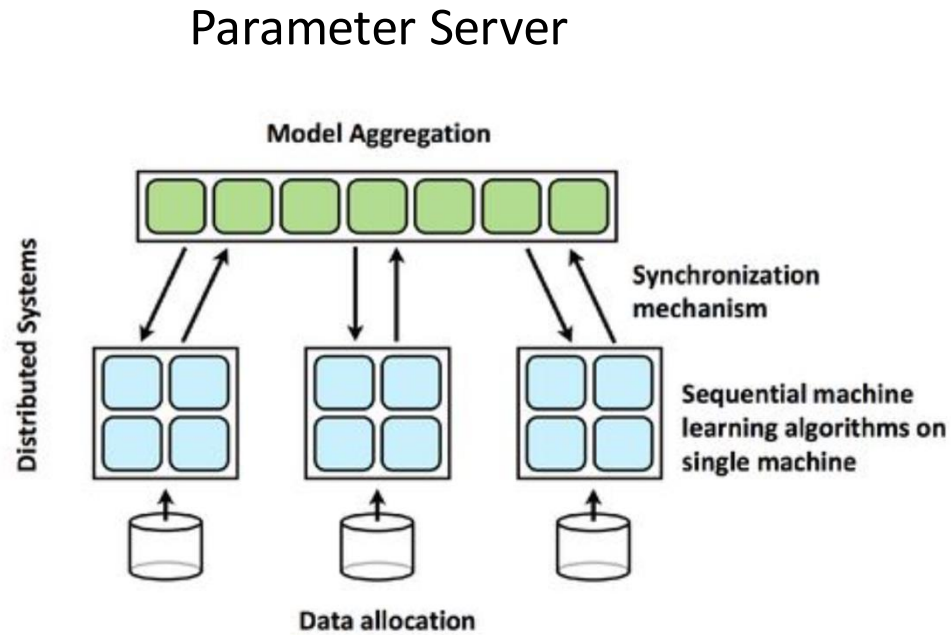
Recall: Data Parallelism



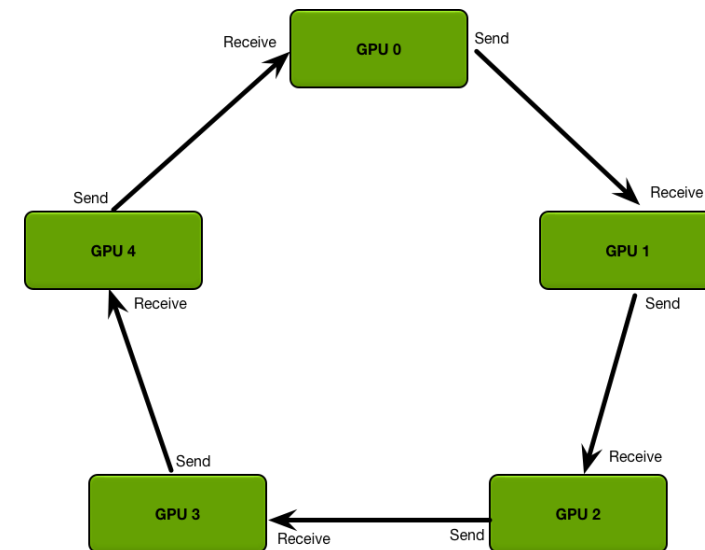
- Partitions a training minibatch across multiple devices
- Linearly scalable

Data parallel training with 2 compute nodes





AllReduce



Data parallelism does not help with excessive model size

- **Tensor Slicing Model Parallelism**
- **Second panel discussion**

LlamaRL: A Distributed Asynchronous Reinforcement Learning Framework for Efficient Large-scale LLM Training

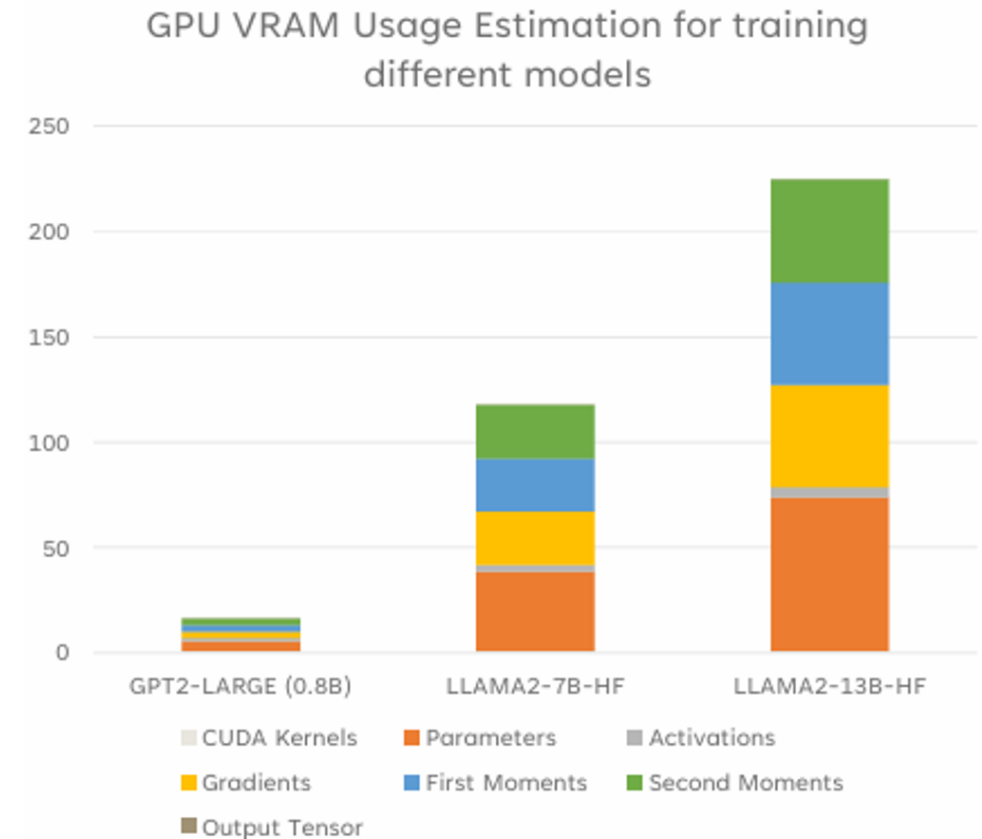
Meta GenAI¹

Large Models Require Large Memory



- Training >> Inference
- Adam >> SGD (*due to optimizer state*)
- Larger minibatch
- More parameters
- ...

Question: How do we enable or even speed up training?

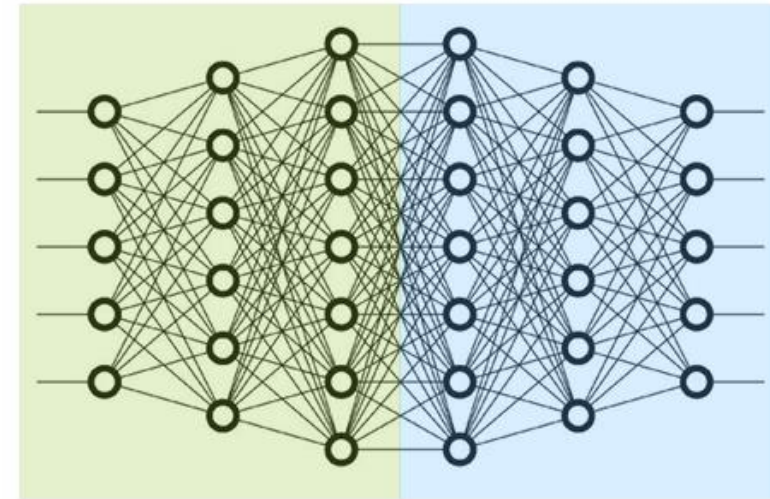


- Paper: <https://arxiv.org/abs/1909.08053>
- Repo: <https://github.com/NVIDIA/Megatron-LM>
- NVIDIA's framework, released in 2019, for efficiently training large language models



Inter-layer (Pipeline) parallelism

- Split sets of layers across multiple workers
- Layer 0, 1, 2 and layer 3, 4, 5 are on different devices

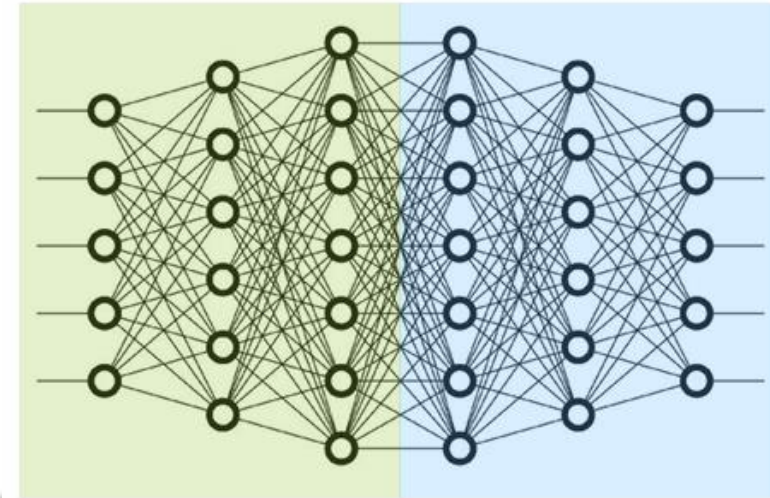


Pipeline Parallelism and Tensor Parallelism



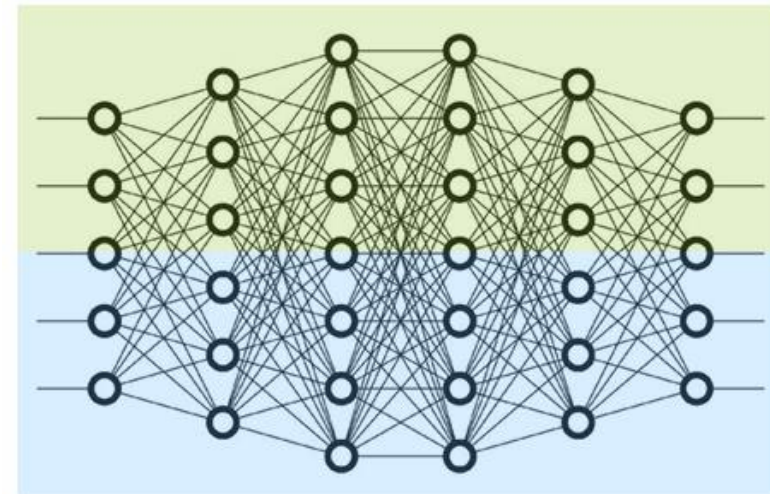
Inter-layer (Pipeline) parallelism

- Split sets of layers across multiple workers
- Layer 0, 1, 2 and layer 3, 4, 5 are on different devices

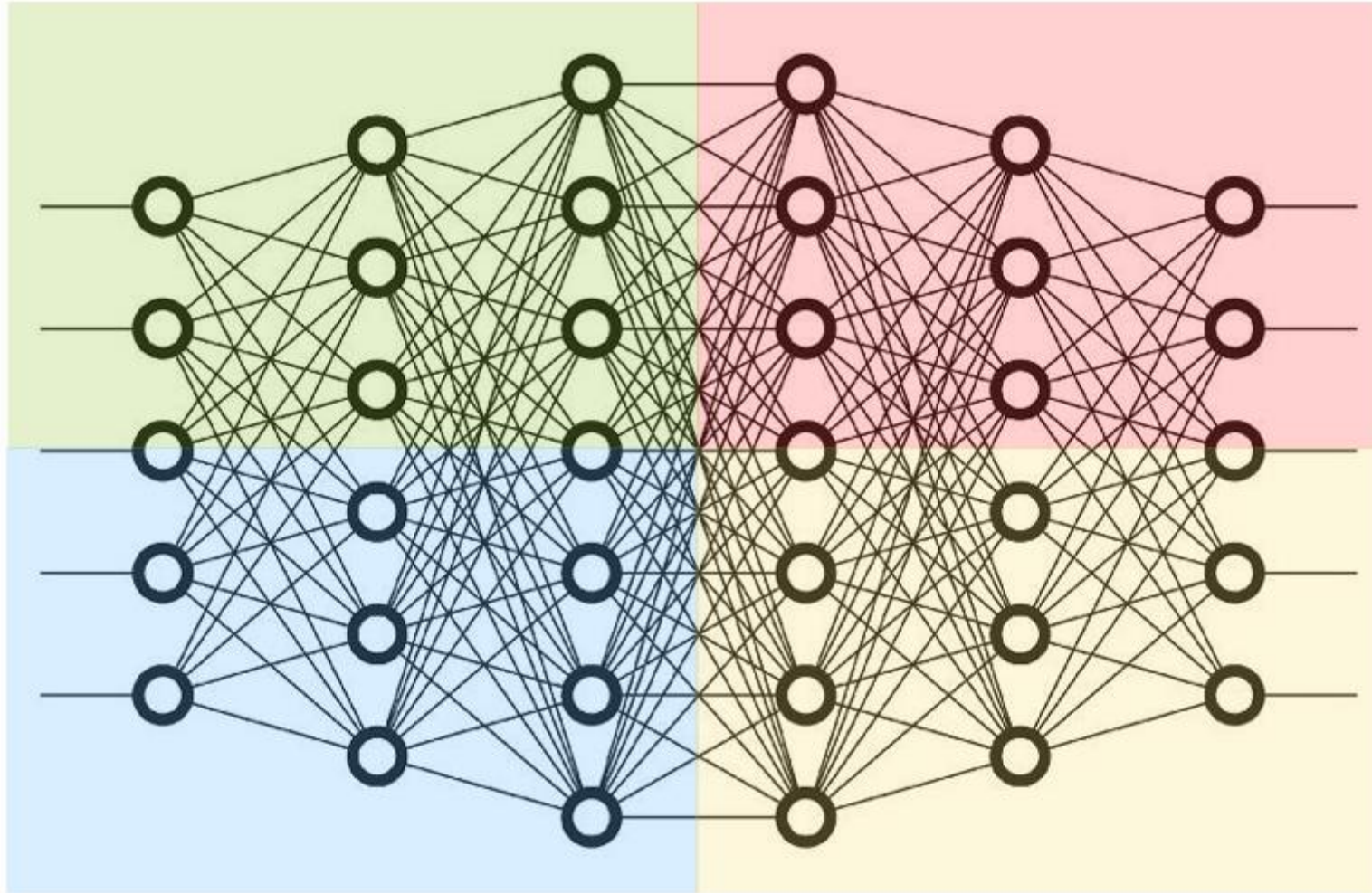


Intra-layer (Tensor) parallelism

- Split individual layers across multiple workers
- Both devices compute different parts of layer 0, 1, 2, 3, 4, 5



Complementary Types of Model Parallelism



Inter + Intra Layer Parallelism

*Row-wise sharding
(of A):*

$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} (A_0 \quad A_1) + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} (A_2 \quad A_3)$$
$$= \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$

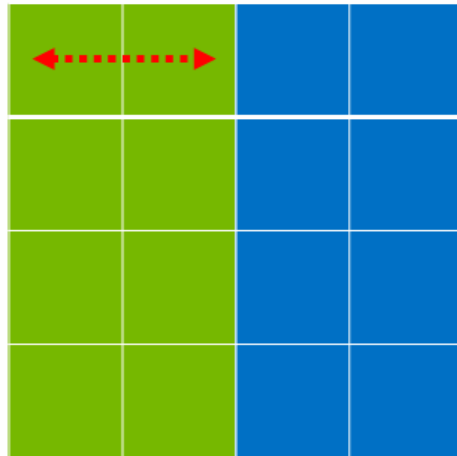
Row-wise sharding (of A):

$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} (A_0 \ A_1) + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} (A_2 \ A_3)$$
$$= \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$

Column-wise sharding (of A):

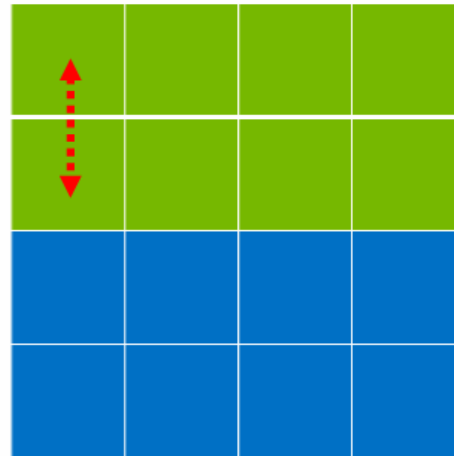
$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \left(\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 \\ A_2 \end{pmatrix} \quad \begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_1 \\ A_3 \end{pmatrix} \right)$$
$$= \underbrace{(X A_A \quad X A_B)}_{\text{Concatenation}}$$

*Row-wise
sharding:*



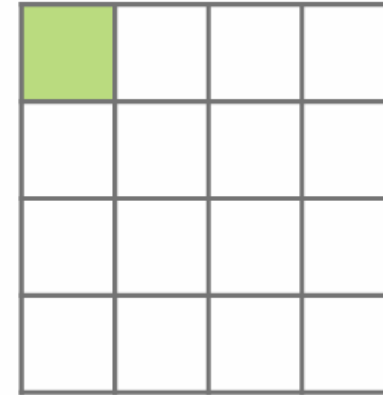
$$X = [X_1, X_2]$$

×



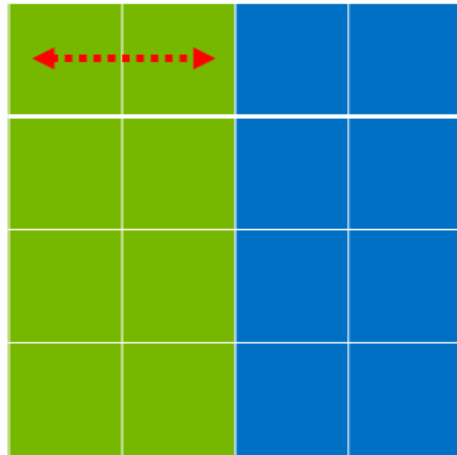
$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

=



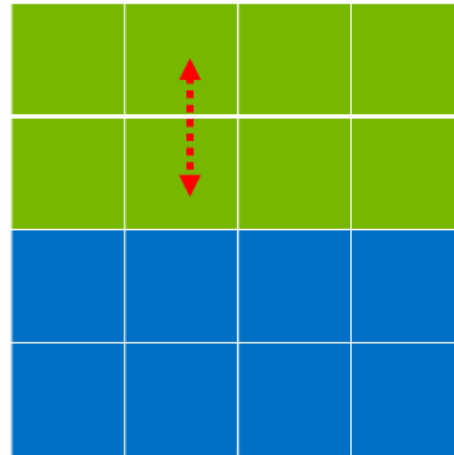
$$Y = Y_1 + Y_2$$

*Row-wise
sharding:*



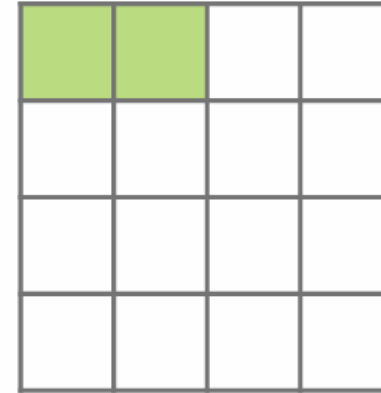
$$X = [X_1, X_2]$$

×



$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

=



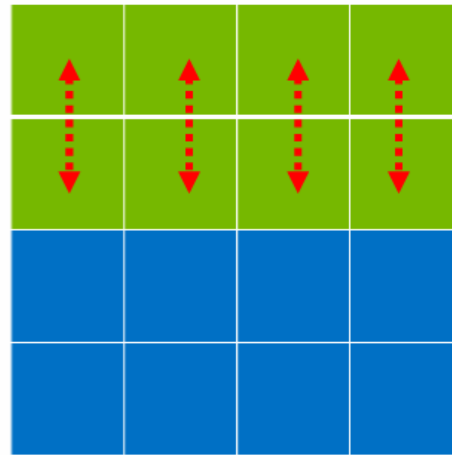
$$Y = Y_1 + Y_2$$

*Row-wise
sharding:*



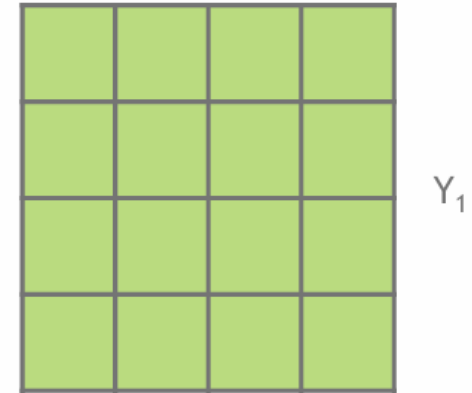
$$X = [X_1, X_2]$$

$\times$



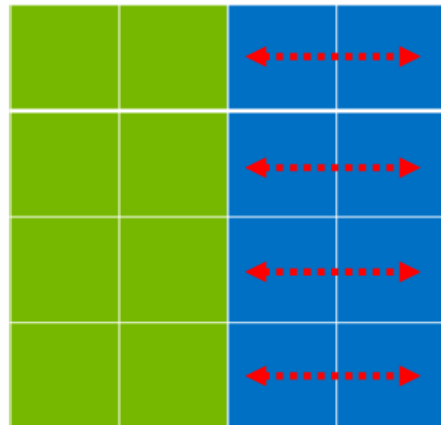
$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$=$



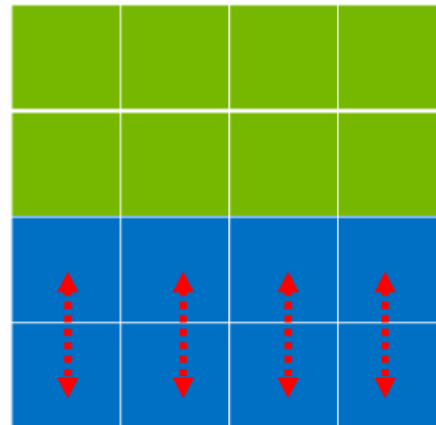
$$Y = Y_1 + Y_2$$

*Row-wise
sharding:*



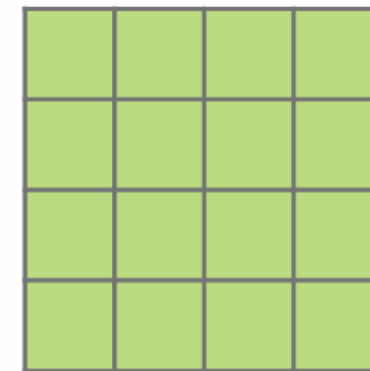
$$X = [X_1, X_2]$$

$\times$



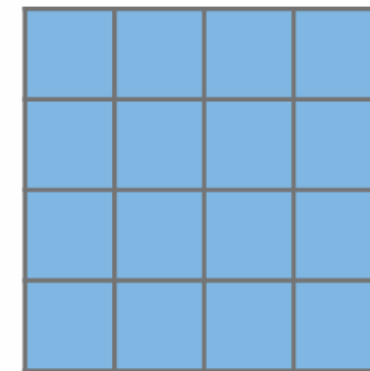
$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$=$



Y_1

$+$



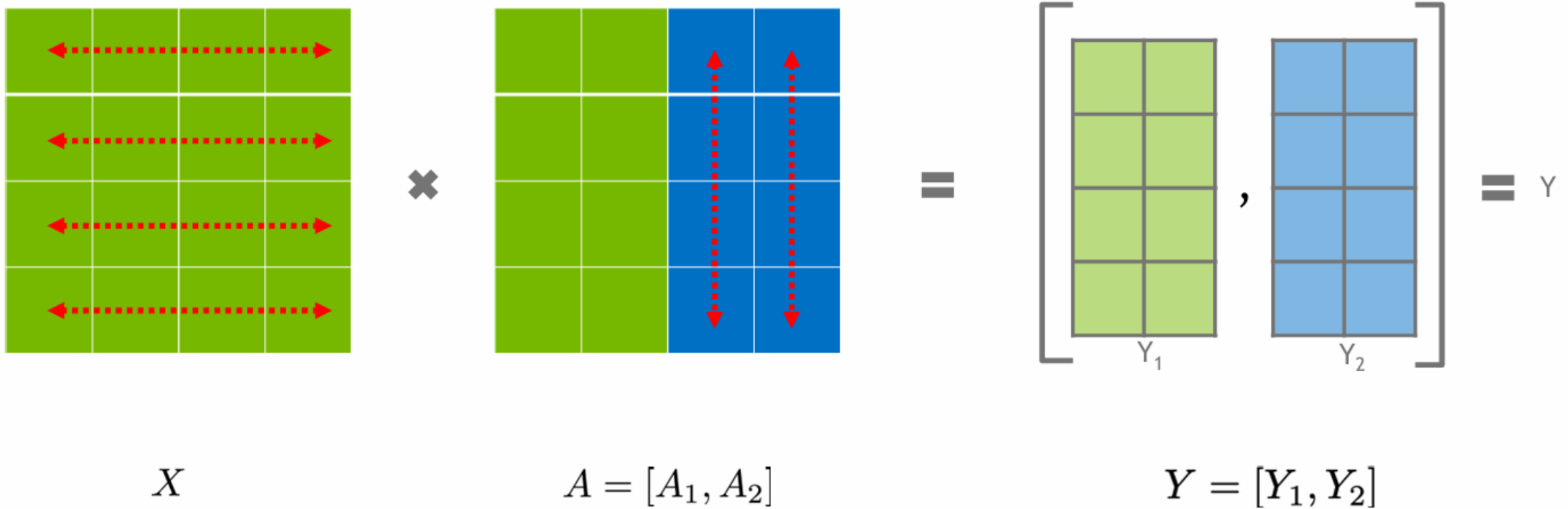
Y_2

$=$

Y

$$Y = Y_1 + Y_2$$

*Column-wise
sharding:*



*Row-wise
sharding:*

$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} (A_0 \ A_1) + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} (A_2 \ A_3)$$
$$= \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$

*Column-wise
sharding:*

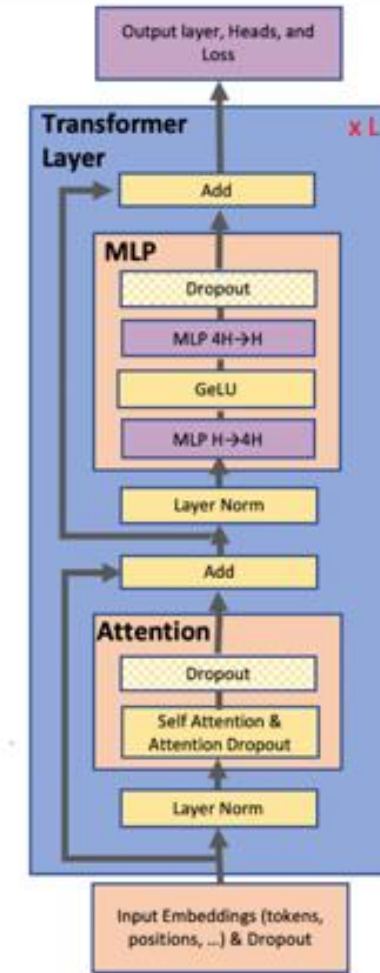
$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \left(\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 \\ A_2 \end{pmatrix} \quad \begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_1 \\ A_3 \end{pmatrix} \right)$$
$$= \underbrace{(X A_A \quad X A_B)}_{\text{Concatenation}}$$

*Column- or row-
wise highly
depends on tensor
operations*

Walk Through Transformers

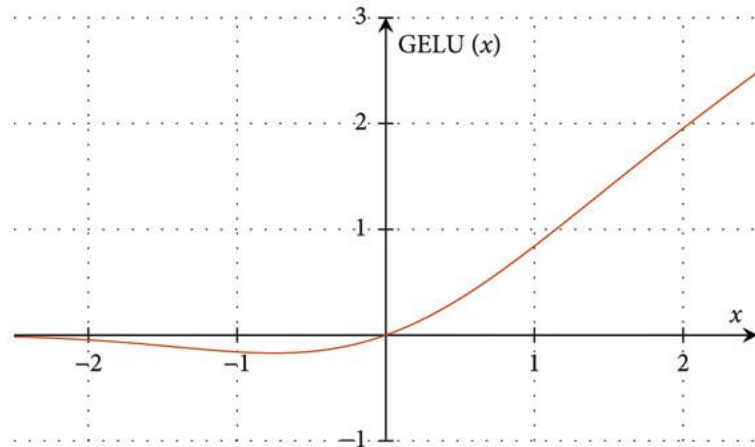


- Transformer layer
 - Self-attention block
 - Two-layer MLP
- Targets:
 - Reduce synchronization cost
 - Simple to implement with a few highly optimized collectives NCCL provides

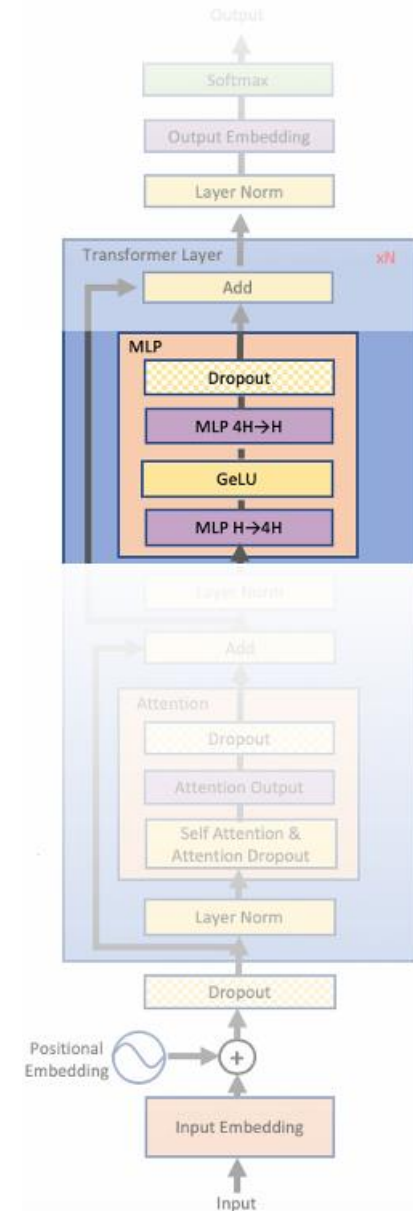


► MLP:

$$Y = \text{GeLU}(XA)$$
$$Z = \text{Dropout}(YB)$$



Dropout (YB):
Randomly drop a certain percentage of values to prevent overfitting



► MLP:

$$Y = \text{GeLU}(XA)$$

$$Z = \text{Dropout}(YB)$$

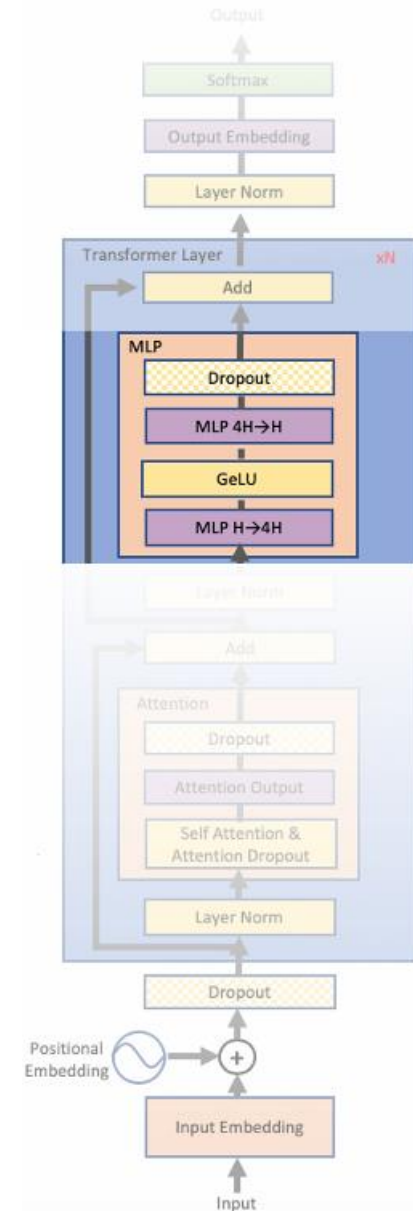
► Approach 1: split X column-wise and A row-wise

$$X = [X_1, X_2] \quad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \longrightarrow Y = \text{GeLU}(X_1A_1 + X_2A_2)$$

GeLU of sums != sum of GeLUs

$$\begin{pmatrix} X_0 \\ X_2 \end{pmatrix} \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} (A_0 \quad A_1) + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} (A_2 \quad A_3)$$

$$= \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$



► MLP:

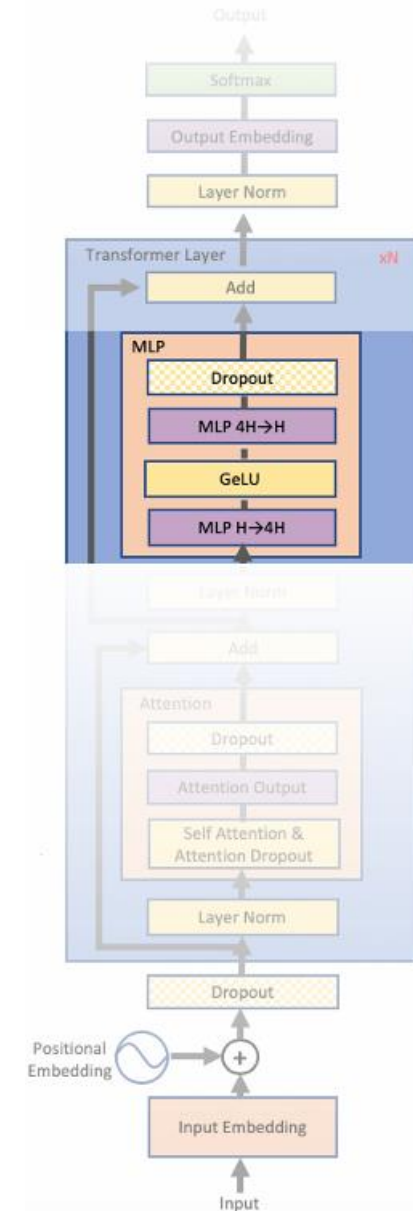
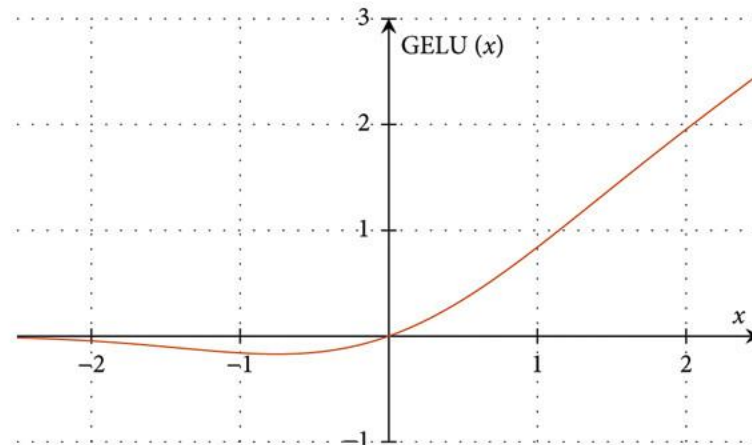
$$Y = \text{GeLU}(XA)$$
$$Z = \text{Dropout}(YB)$$

- Approach 1: split X column-wise and A row-wise

$$X = [X_1, X_2] \quad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \quad \longrightarrow \quad Y = \text{GeLU}(X_1 A_1 + X_2 A_2)$$

GeLU of sums != sum of GeLUs

Requires AllReduce
before GeLU



► MLP:

$$Y = \text{GeLU}(XA)$$

$$Z = \text{Dropout}(YB)$$

► Approach 1: split X column-wise and A row-wise

$$X = [X_1, X_2] \quad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \longrightarrow Y = \text{GeLU}(X_1 A_1 + X_2 A_2)$$

- Requires synchronization before GeLU

GeLU of sums != sum of GeLUs

► Approach 2: split A column-wise

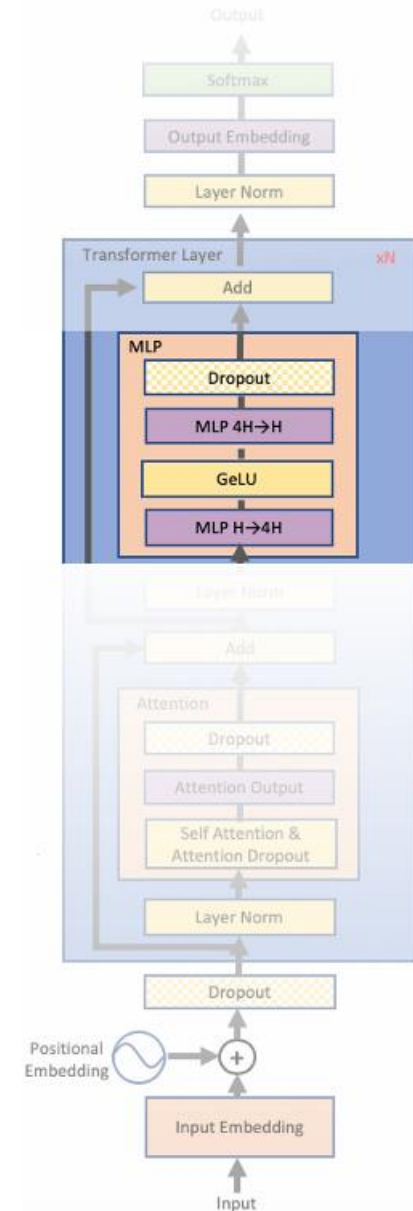
$$A = [A_1, A_2] \longrightarrow [Y_1, Y_2] = [\text{GeLU}(X A_1), \text{GeLU}(X A_2)]$$

- No synchronization necessary

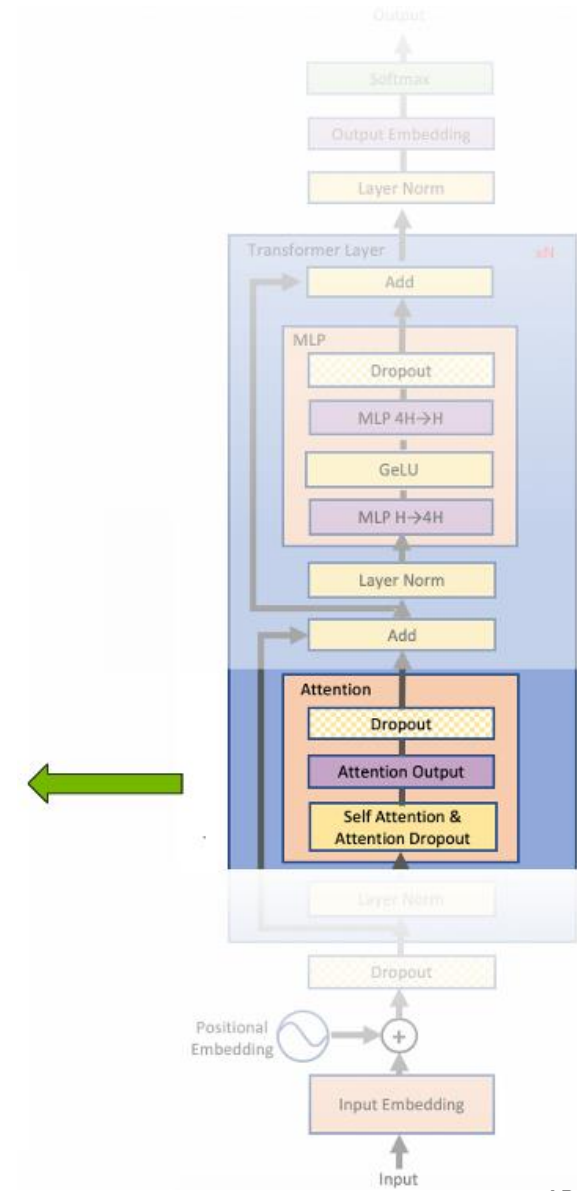
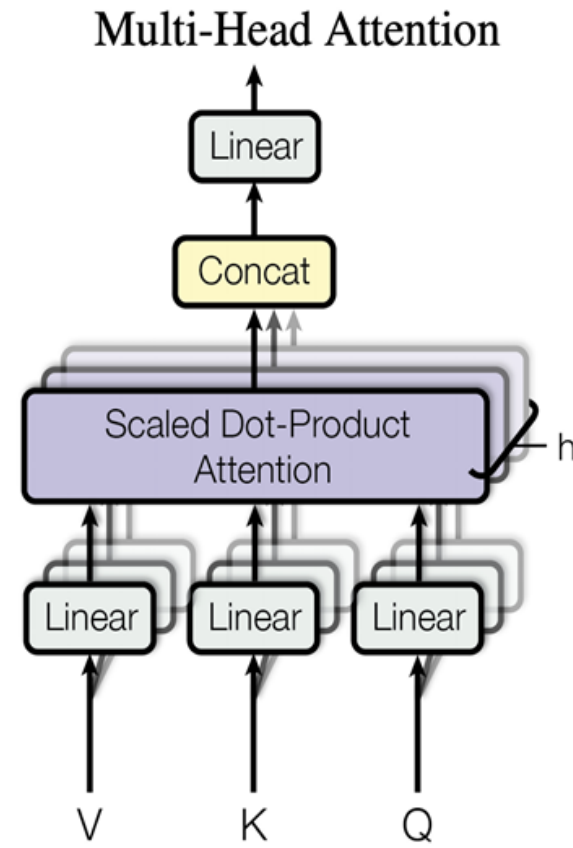
Can run in parallel now!

$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \left(\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 \\ A_2 \end{pmatrix} \quad \begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_1 \\ A_3 \end{pmatrix} \right)$$

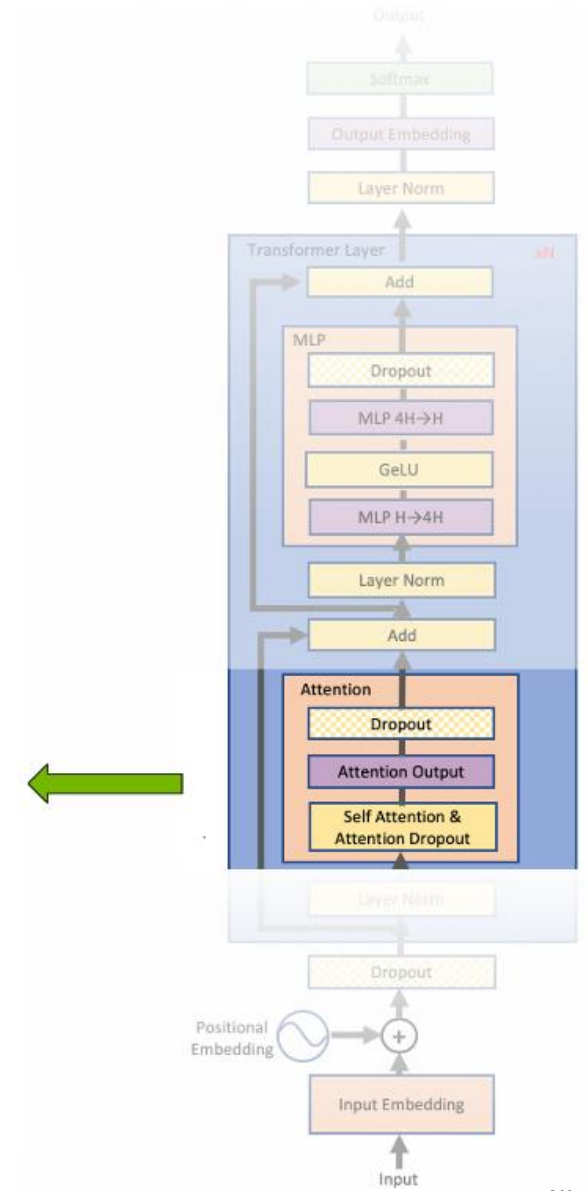
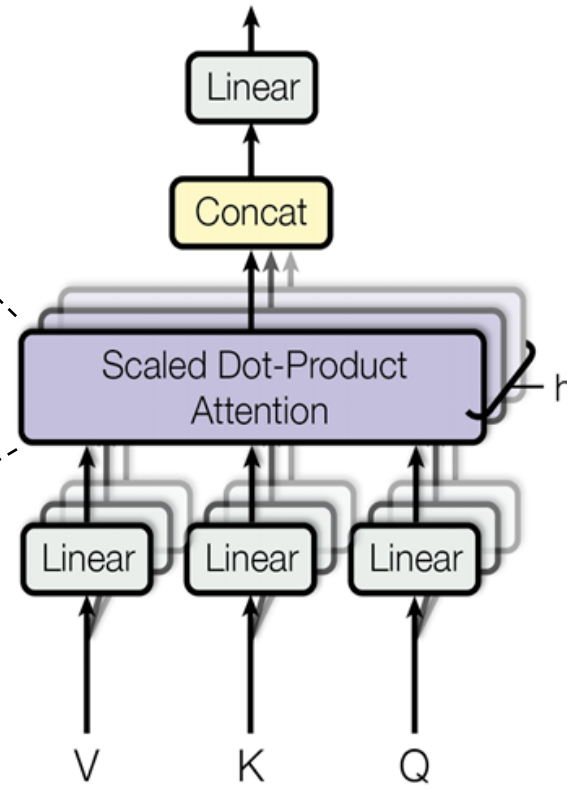
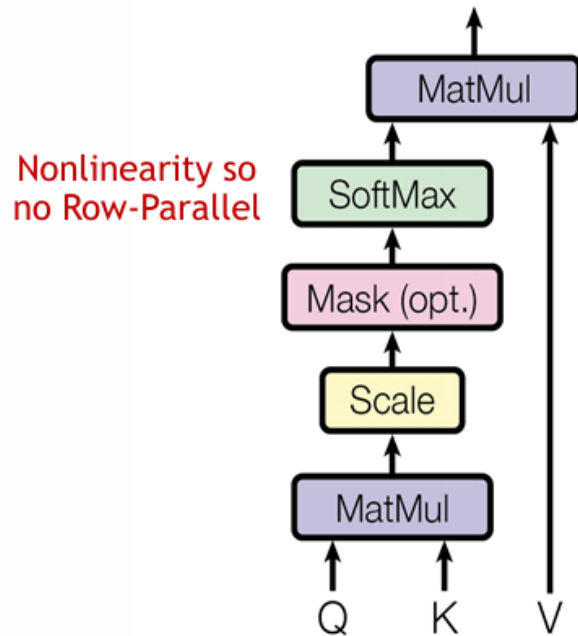
$$= \underbrace{(X A_A \quad X A_B)}_{\text{Concatenation}}$$

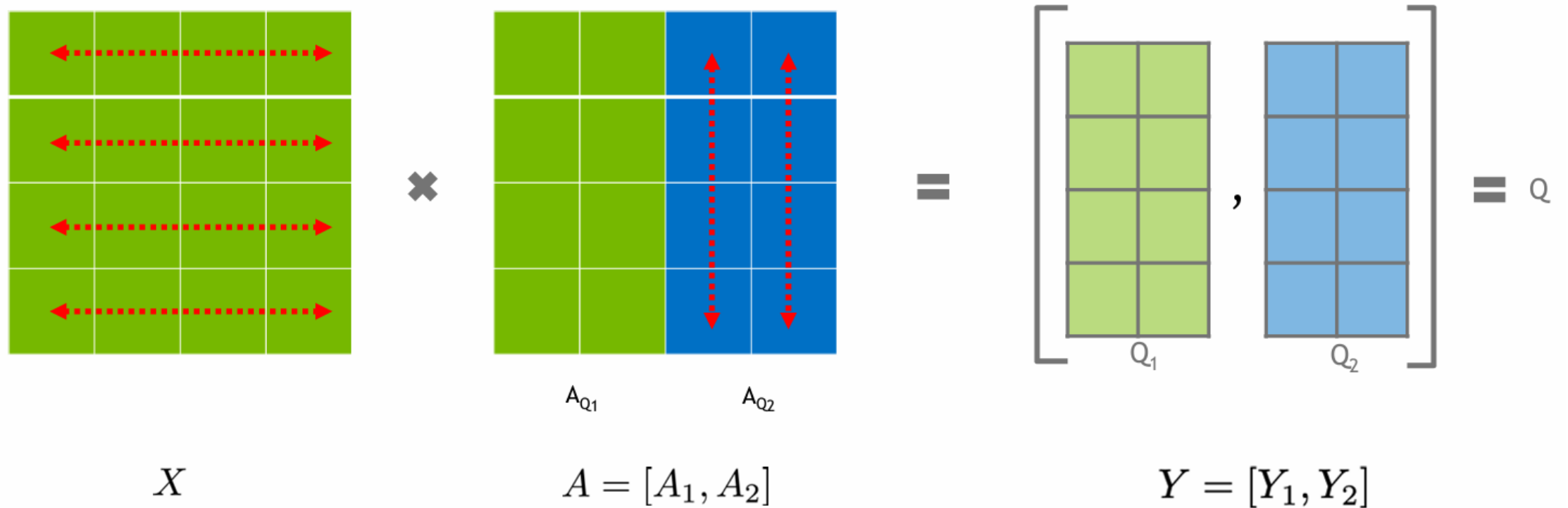


Tensor Parallelism: Self-Attention



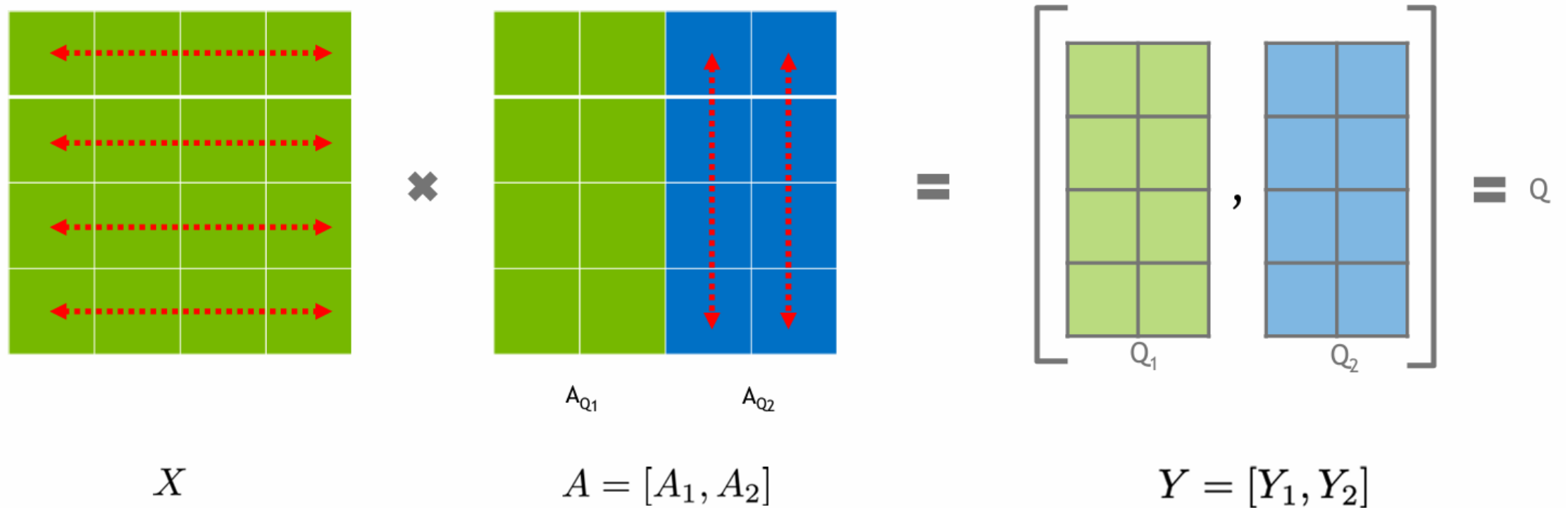
Tensor Parallelism: Self-Attention





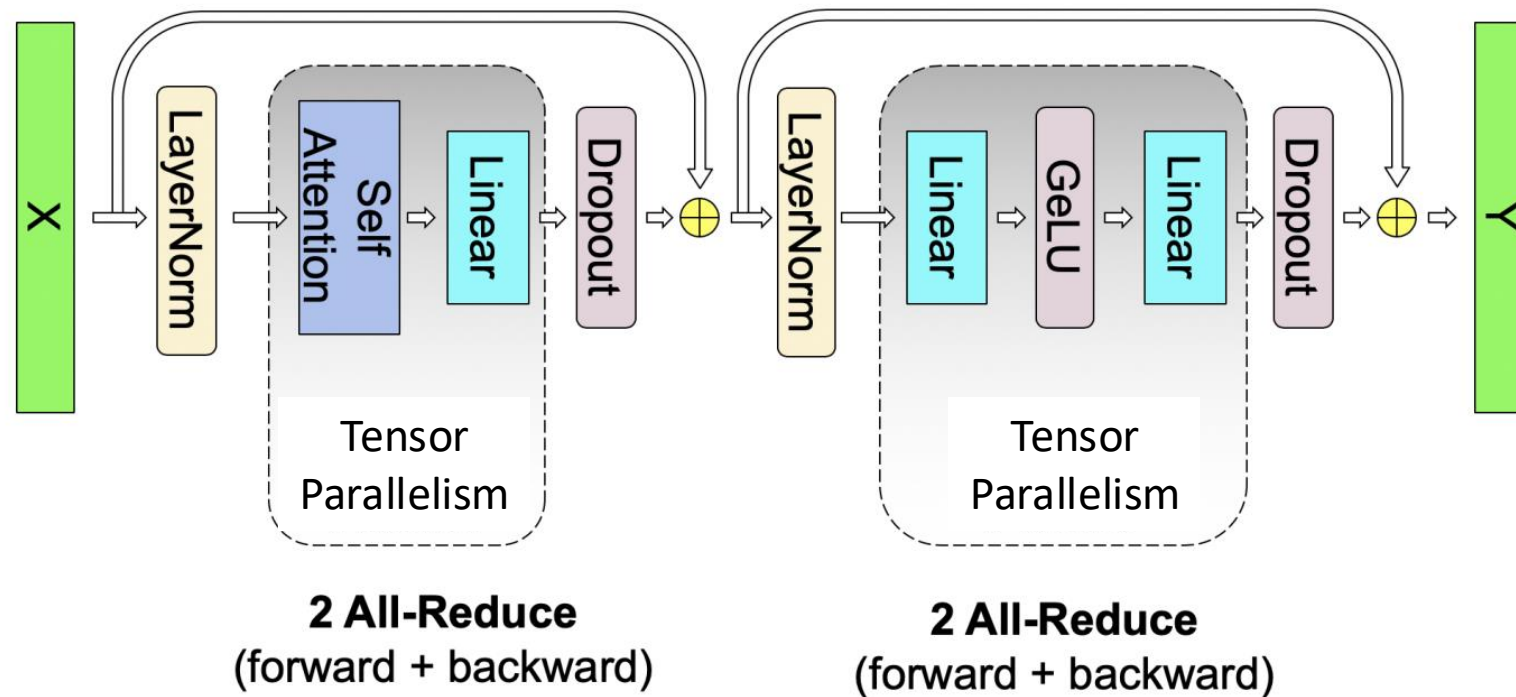
Attention heads can be parallelized with Column Parallel GEMMs (ex. Query head 1 (Q_1) and Query head 2 (Q_2))

Tensor Parallelism: Self-Attention

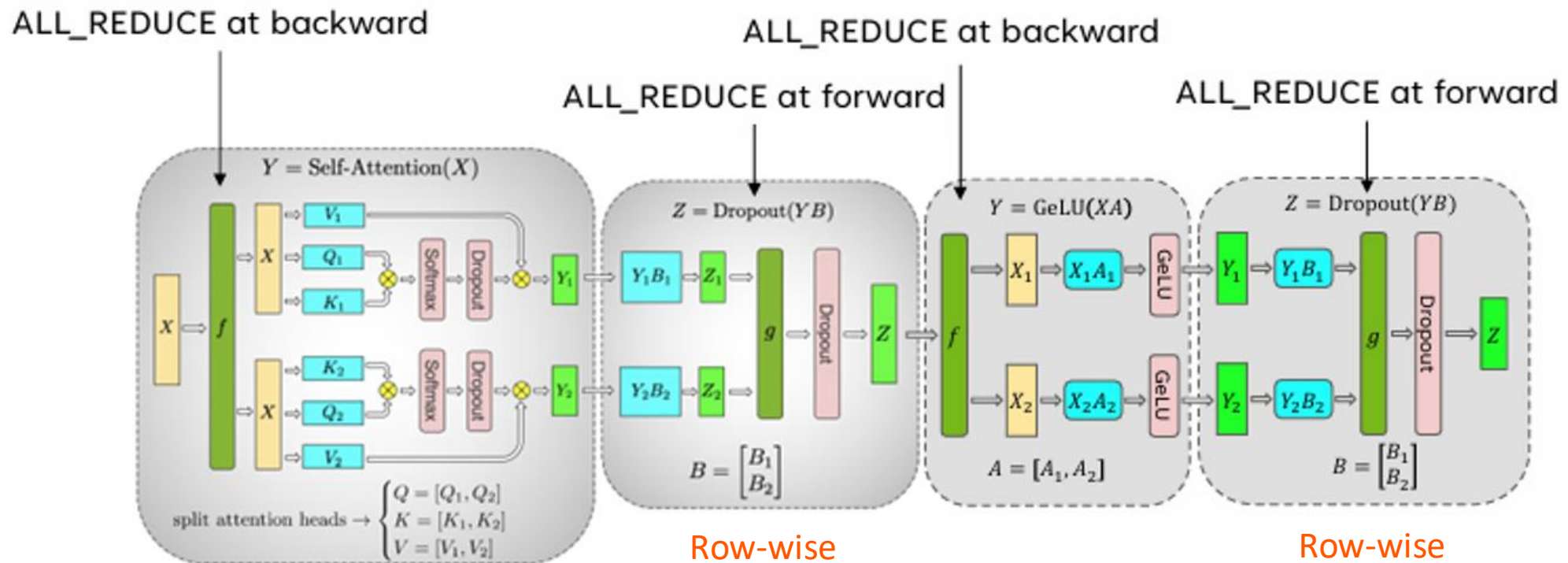


Attention heads can be parallelized with Column Parallel GEMMs (ex. Query head 1 (Q_1) and Query head 2 (Q_2))

- 4 total communication operations in 1 forward + backward pass

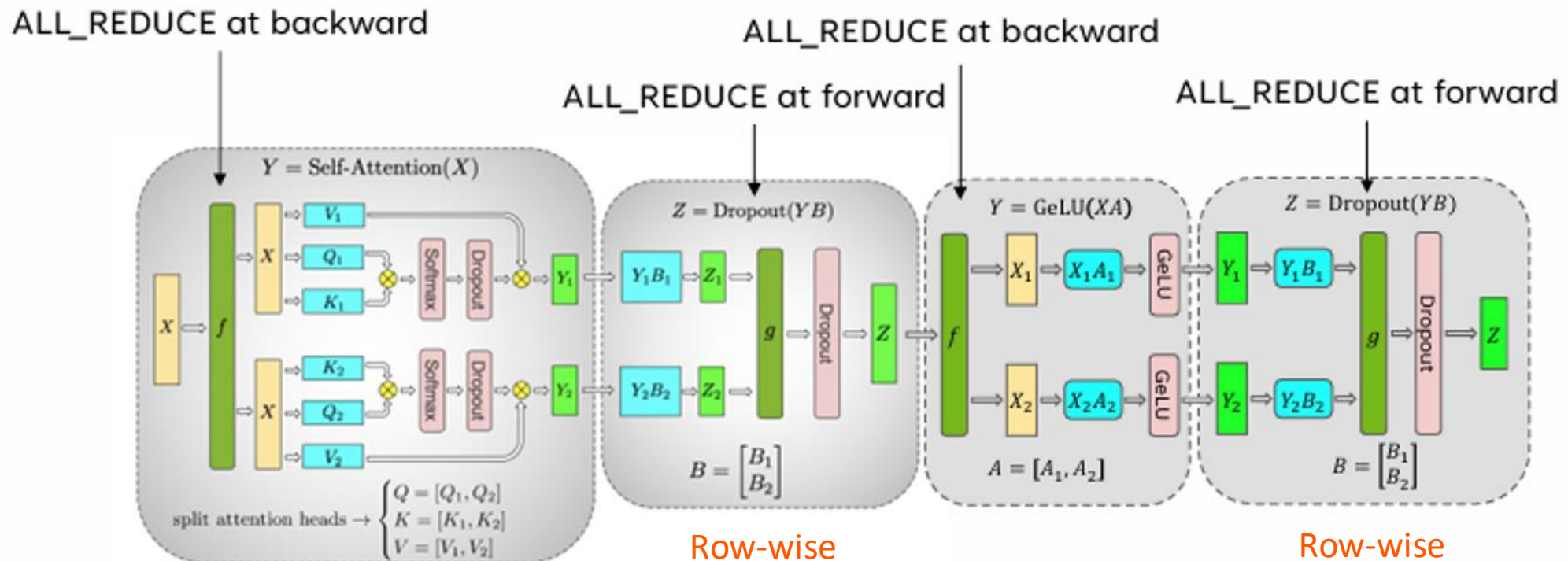


- 4 total communication operations in 1 forward + backward pass



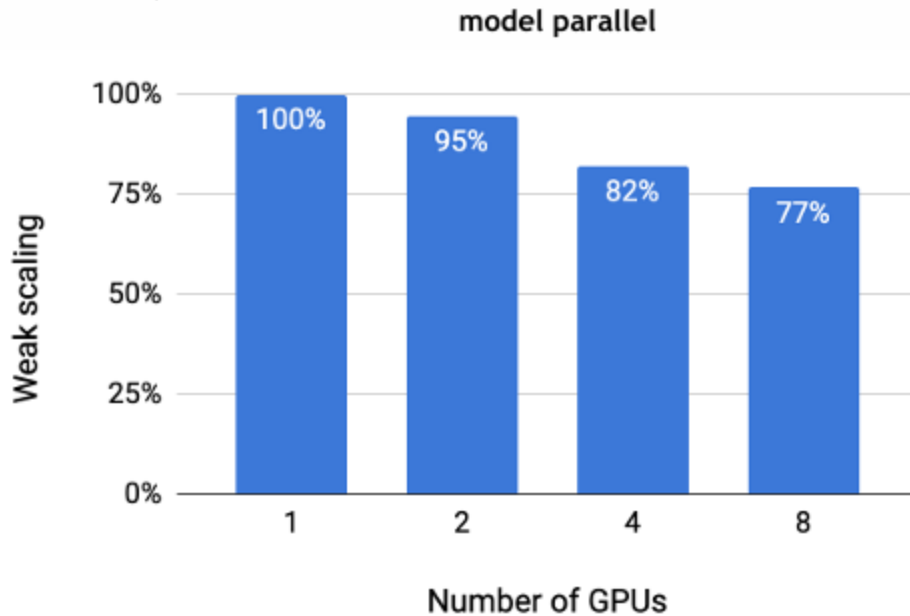
$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} (A_0 \ A_1) + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} (A_2 \ A_3) \\ = \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$

- 4 total communication operations in 1 forward + backward pass



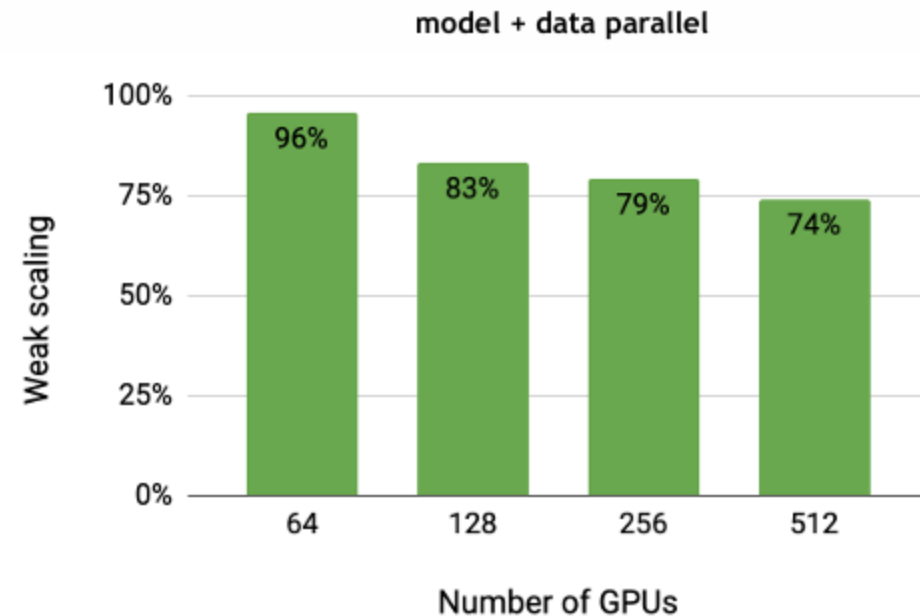
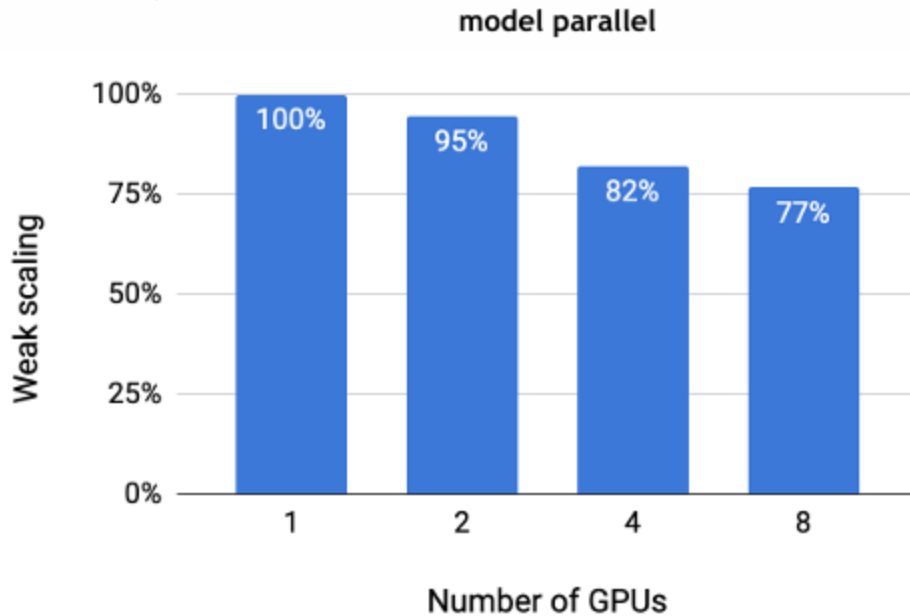
Question: What if we have more GPUs than the number of heads?

Weak-scaling: the effective execution time



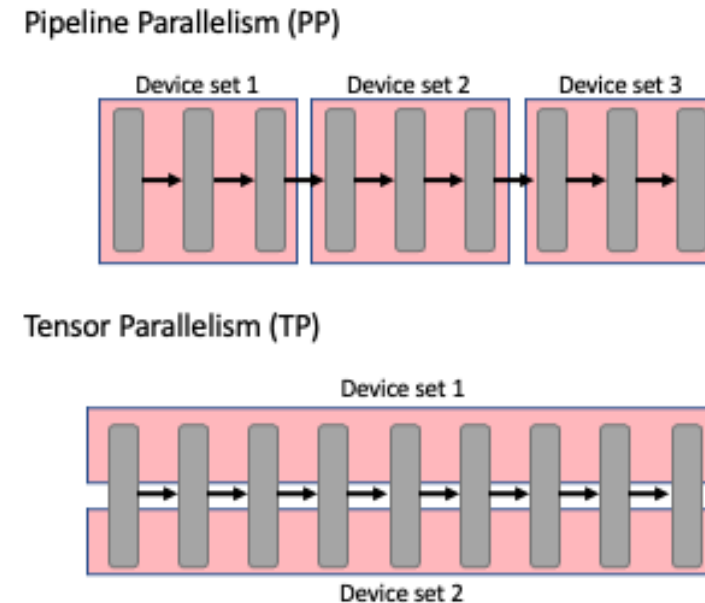
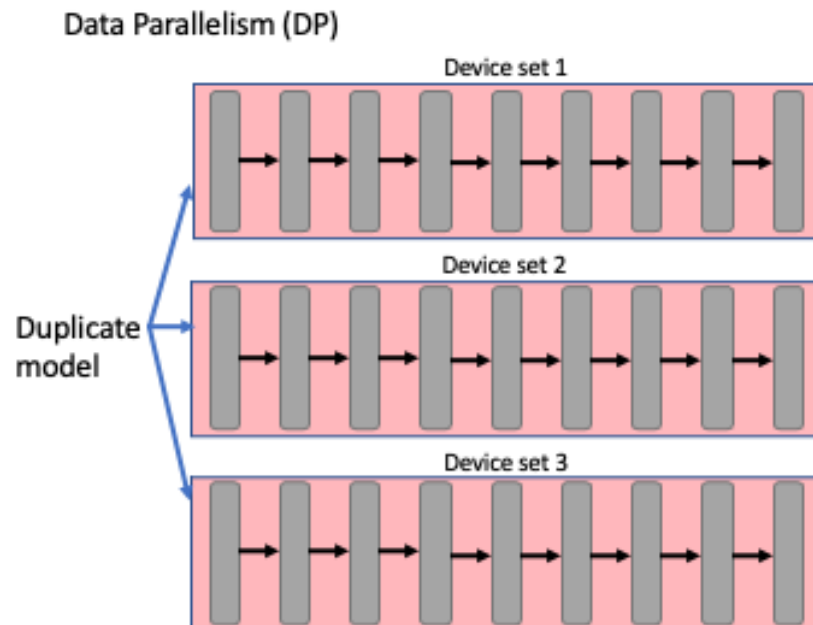
Baseline (1.2B parameters on a single GPU) achieves **39 TeraFLOPs per second**, i.e. **30% of the theoretical peak** during the entire training process

Weak-scaling: the effective execution time



Baseline (1.2B parameters on a single GPU) achieves **39 TeraFLOPs per second**, i.e. **30% of the theoretical peak** during the entire training process

- Data parallelism assumes hosting the full model within a unit
- Tensor parallelism (TP): split matrix row-wise or column-wise
- Reduce synchronization cost is crucial as TP is communication-intensive



LlamaRL: A Distributed Asynchronous Reinforcement Learning Framework for Efficient Large-scale LLM Training

Meta GenAI¹

Introduce how Meta team performs efficient RLHF training, with asynchronous execution across models (e.g., actor and reward models)

Questions?

Normal

Operation: $Y_{n \times n} = X_{n \times n} A_{n \times n}$

Flops: $2n^3$

Bandwidth: $6n^2$

Intensity: $\frac{1}{3}n$

Parallel

Operation: $Y_{n \times (n/p)} = X_{n \times n} A_{n \times (n/p)}$ (P: # of workers)

Flops: $2n^3/p$

Bandwidth: $2n^2(1 + 2/p)$

Intensity: $\frac{1}{2+p}n$

Complexity Analysis of Tensor Parallelism



Normal

Operation: $Y_{n \times n} = X_{n \times n} A_{n \times n}$

Flops: $2n^3$

Bandwidth: $6n^2$

Intensity: $\frac{1}{3}n$

Parallel

Operation: $Y_{n \times (n/p)} = X_{n \times n} A_{n \times (n/p)}$

Flops: $2n^3/p$

Bandwidth: $2n^2(1 + 2/p)$

Intensity: $\frac{1}{2+p}n$

(P: # of workers)

Ratio between serial and parallel

Flops: $1/p$ 

Bandwidth: $\frac{1 + 2/p}{3}$ 

Intensity: $\frac{3}{2+p}$ 